

SCRIBE

Loop Engineering in Long Horizon Tasks Needs Agent to Write What They Did Each Turn

周浩洋 · Jilin University (JLU), School of Software (SE)

Research Assistant · 2025.04.17 – Present

目录

01

training-free 方法

不可能实现 loop engineering 的
token 效率提升

02

training 方法 · ReAct

使用 ReAct 进行 loop engineering

03

ReAct 的问题

使用 ReAct 进行 loop engineering 的
问题

04

training 方法 · SCRIBE

使用 SCRIBE 进行 loop engineering

05

SCRIBE 上下文管理机制

细述 turn 级上下文与 KV-cache 复用

06

reward mechanism of a
turn

16 个 metric 与权重配比

07

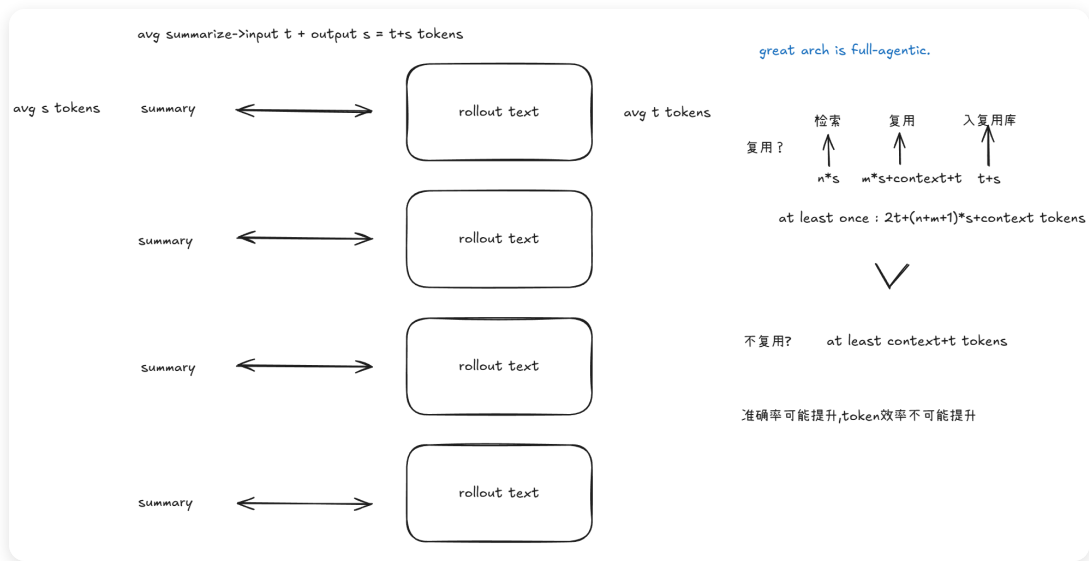
abstract of our contribution

小改动 → 上下文机制变化 → 衍生收益

01 · training-free 方法

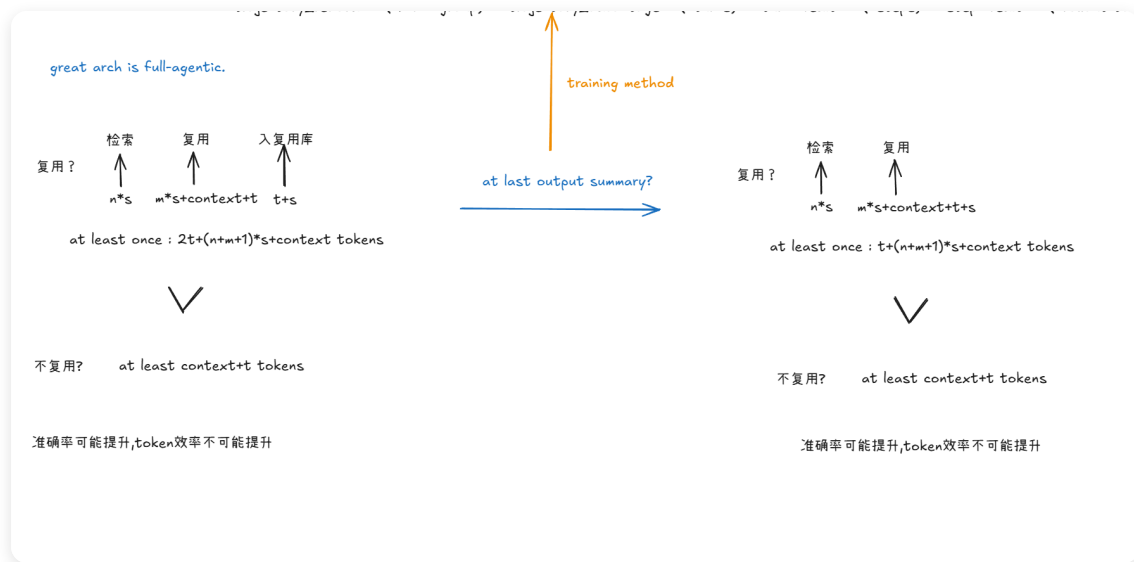
不可能实现 loop engineering 的 token 效率提升

01 · training-free 方法 — 不可能实现 loop engineering 的 token 效率提升



training-free 路径下, loop engineering 的 token 效率提升瓶颈 (一)

01 · training-free 方法 — 不可能实现 loop engineering 的 token 效率提升

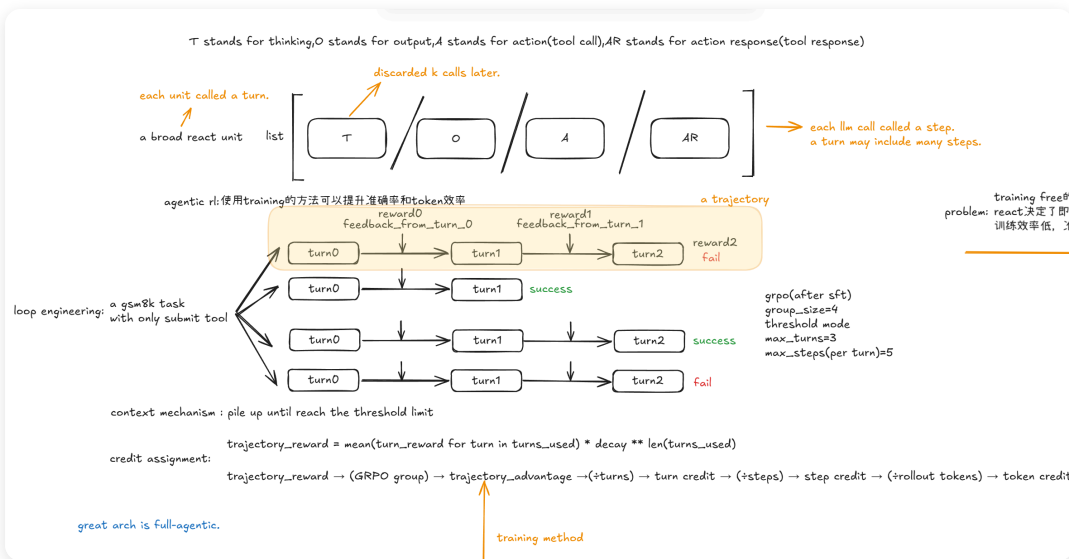


training-free 路径下, loop engineering 的 token 效率提升瓶颈 (二)

02 · training 方法

使用 ReAct 进行 loop engineering

02 · training 方法 — 使用 ReAct 进行 loop engineering



broad ReAct: 经典 Think→Act→Observe 循环作为 loop engineering 的训练范式

03 · ReAct 的问题

使用 ReAct 进行 loop engineering 的问题

03 · training 方法 — 使用 ReAct 进行 loop engineering 的问题

training free的话, 每次试错的经验是一次性的, 否则设计记忆架构, token效率必定增加。
problem: react决定了即使training, 状态和状态转换是隐式的, 对状态转换的后果没有前瞻性和规划, 短视, 所以要不断试错。
训练效率低, 准确率和token效率提高还不够多。且刚才的方法并没有精细的信用分配。

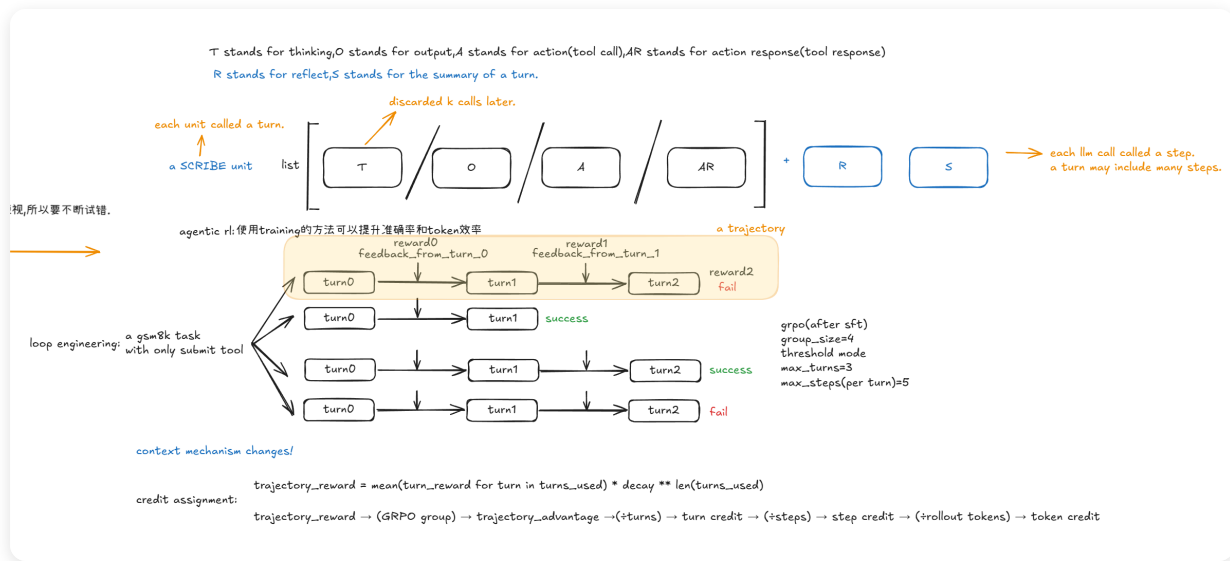


broad ReAct 在长程 loop engineering 中暴露的问题

04 · training 方法 · SCRIBE

ReAct 问题的解法：使用 SCRIBE 进行 loop engineering

04 · training 方法 — 使用 SCRIBE 进行 loop engineering

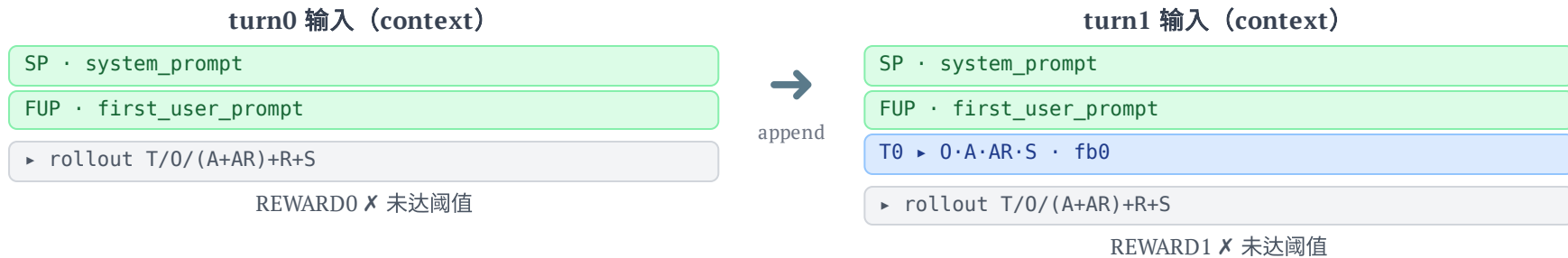


SCRIBE：在 ReAct 基础上为每个 turn 显式写入 Reflect (R) 与 Turn-Summary (S)

05 · SCRIBE 上下文管理机制

细述 turn 级上下文管理：KV-cache 复用、recent-k 保留、自动压缩

05 · SCRIBE 上下文管理机制 - turn0 → turn1



05 · SCRIBE 上下文管理机制 - turn1 → turn2

turn1 输入 (context)

SP · system_prompt

FUP · first_user_prompt

T0 ▶ 0·A·AR·S · fb0

▶ rollout T/0/(A+AR)+R+S

REWARD1 ✕ 未达阈值



append

turn2 输入 (context)

SP · system_prompt

FUP · first_user_prompt

T0 ▶ 0·A·AR·S · fb0

T1 ▶ 0·A·AR·S · fb1

▶ rollout T/0/(A+AR)+R+S

REWARD2 ✕ 未达阈值

前缀 **SP + FUP + T0** 全部复用，仅追加 turn1 留存块 + fb1。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算

05 · SCRIBE 上下文管理机制 - turn2 → turn3

turn2 输入 (context)

SP · system_prompt
FUP · first_user_prompt
T0 ▶ 0·A·AR·S · fb0
T1 ▶ 0·A·AR·S · fb1
▶ rollout T/0/(A+AR)+R+S

REWARD2 ✕ 未达阈值



turn3 输入 (context)

SP · system_prompt
FUP · first_user_prompt
T0 ▶ 0·A·AR·S · fb0
T1 ▶ 0·A·AR·S · fb1
T2 ▶ 0·A·AR·S · fb2
▶ rollout T/0/(A+AR)+R+S

REWARD3 ✕ 未达阈值

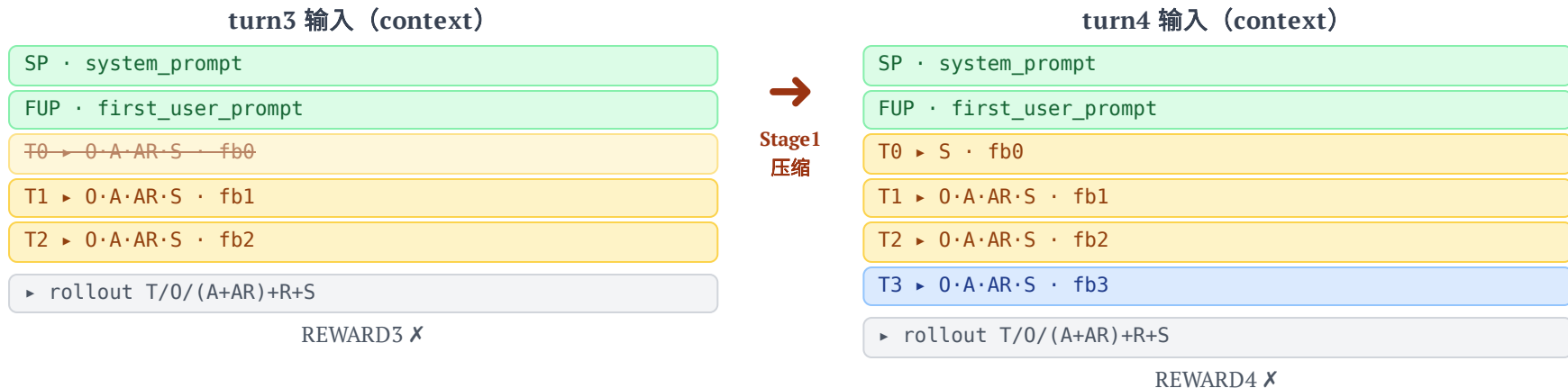
recent-k=3 窗口内逐 turn 追加，整段前缀 SP+FUP+T0+T1 复用，仅新增 T2。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算

05 · SCRIBE 上下文管理机制 - turn3 → turn4（首次压缩）



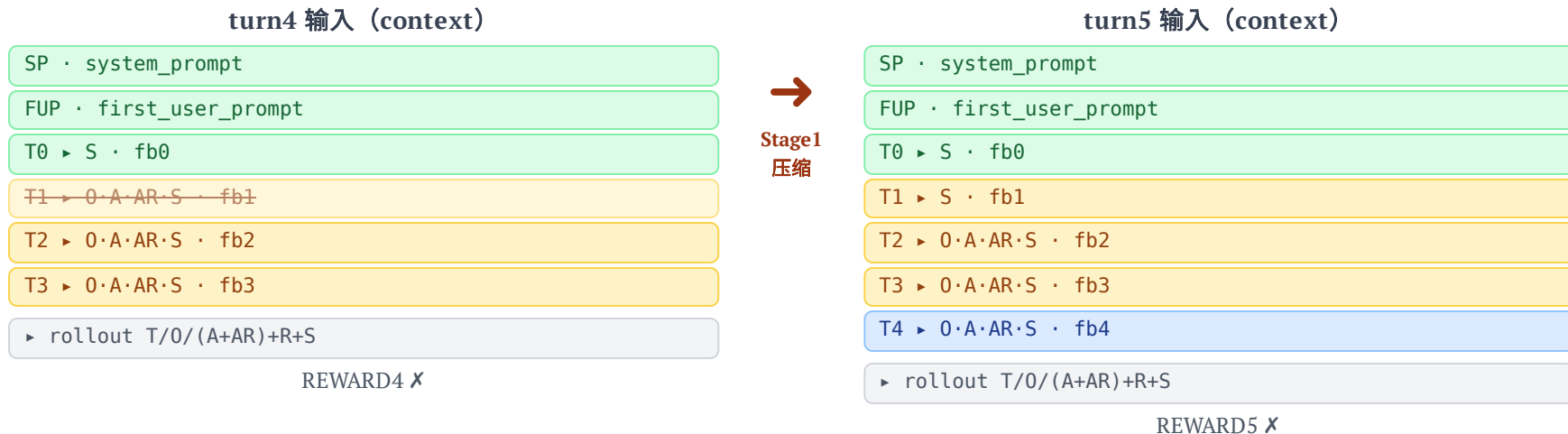
turn0 超出 recent-k=3 被降级为 S (O/A/AR 丢弃, 只留 turn_summary)；公共前缀在 **T0 处断裂** → T1/T2 虽内容不变也须重算。代价是换得后续 append 不再无限增长。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn4 → turn5（压缩推进）



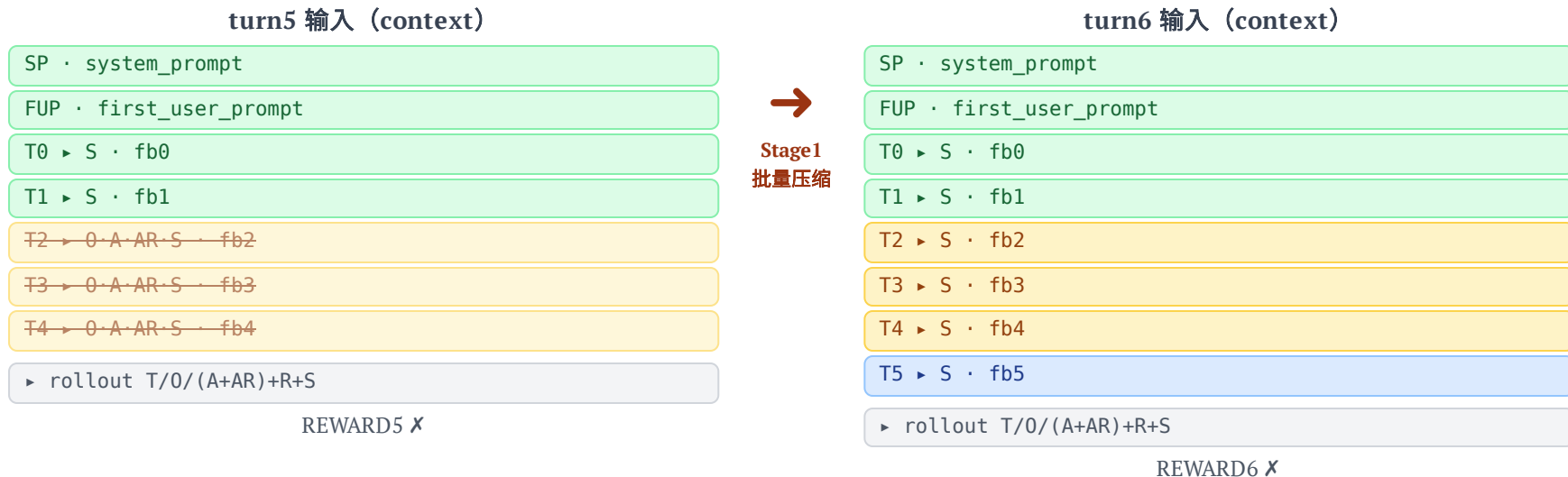
turn1 降级为 S；前缀延至 **T0:S** 后断裂，T2/T3 重算。recent-k=3 窗口（T2/T3/T4）保持全量。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算（删除线=被丢弃）

05 · SCRIBE 上下文管理机制 - turn5 → turn6（批量压缩）



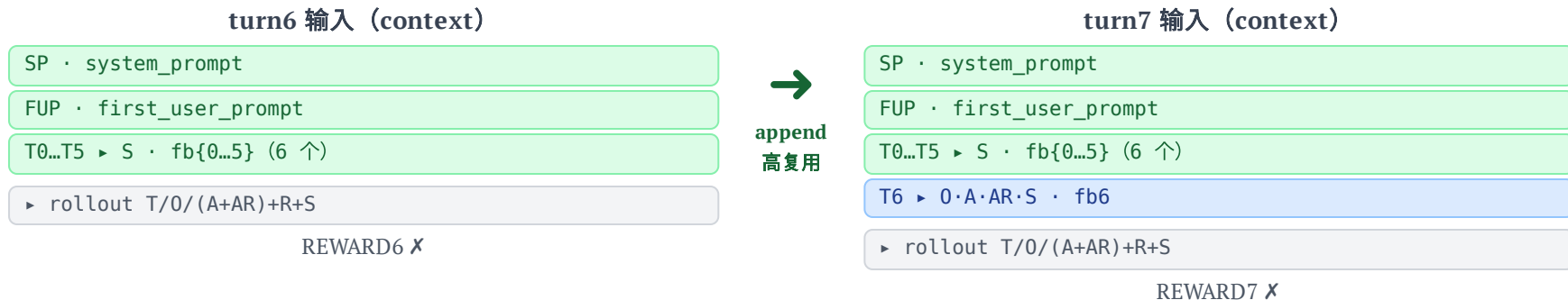
输入逼近上限 → **rollout** 前触发 Stage1 批量压缩：T2/T3/T4 的 O/A/AR 全折叠为 S；前缀在 T2 断裂，T3/T4 重算。原子 turn：压缩只发生在 turn 之间，不中断 turn 内 rollout。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算（删除线=被丢弃）

05 · SCRIBE 上下文管理机制 - turn6 → turn7（高复用）



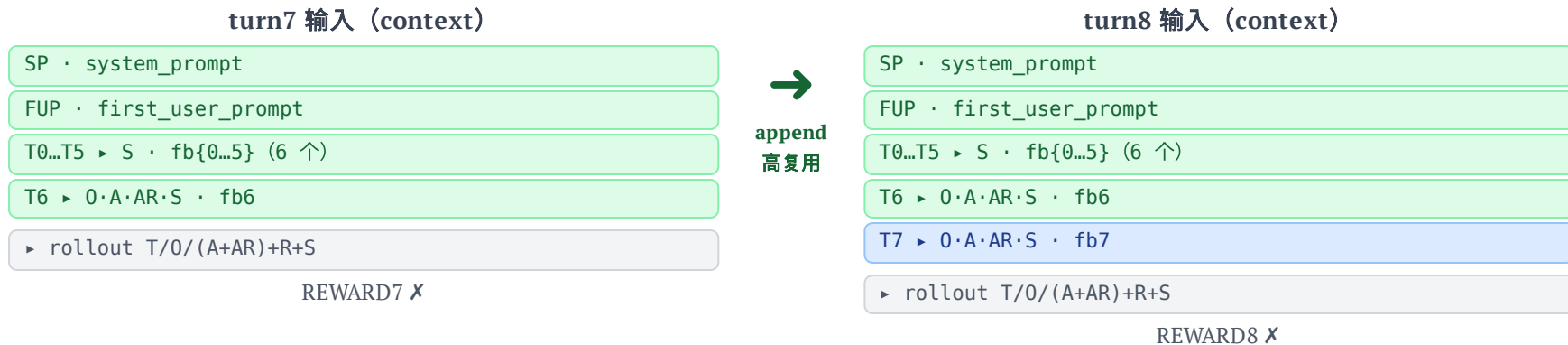
压缩后前缀短而稳定 → **整段 KV-cache 命中**，仅追加 turn6。SCRIBE 的 KV-cache 红利显现：每 turn 只为新增后缀付费。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn7 → turn8（高复用）



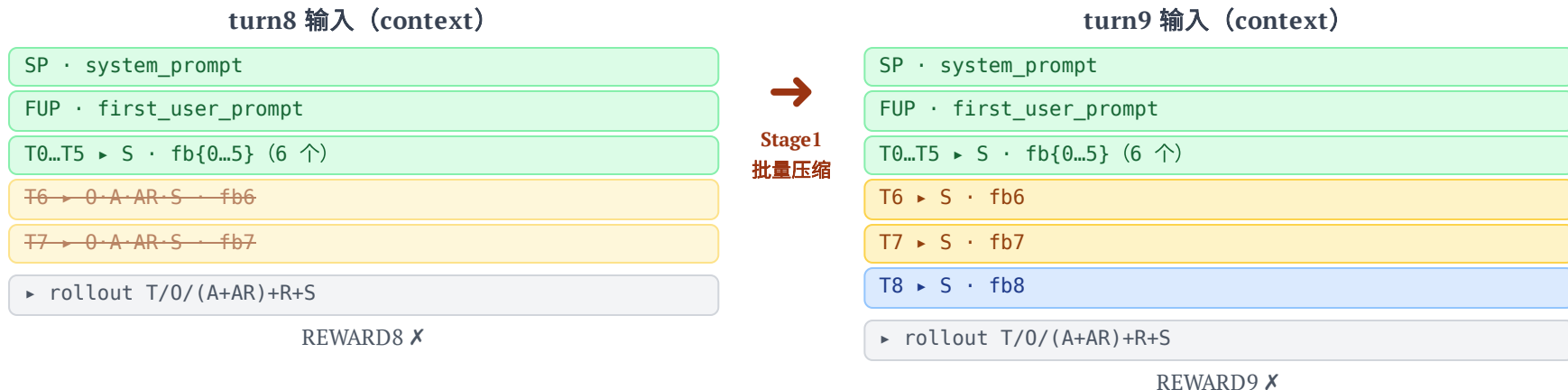
整段复用 + 追加 turn7；前缀越长，复用比例越高，单 turn 增量成本越小。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn8 → turn9（再次批量压缩）



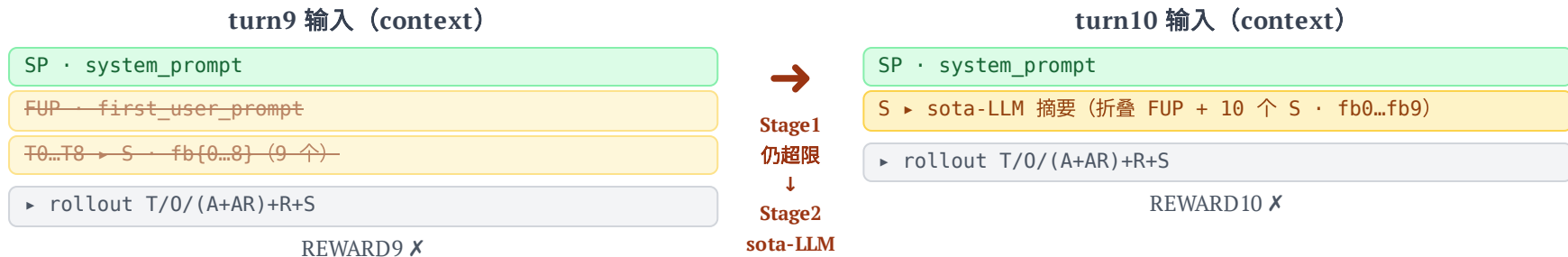
再次逼近上限 → Stage1 压缩 T6/T7 为 S；前缀在 T6 断裂。压缩是周期性的：每当 recent-k 窗口外累积过多 O/A/AR，就折叠一次。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn9 → turn10 (Stage1 → Stage2 双重压缩)



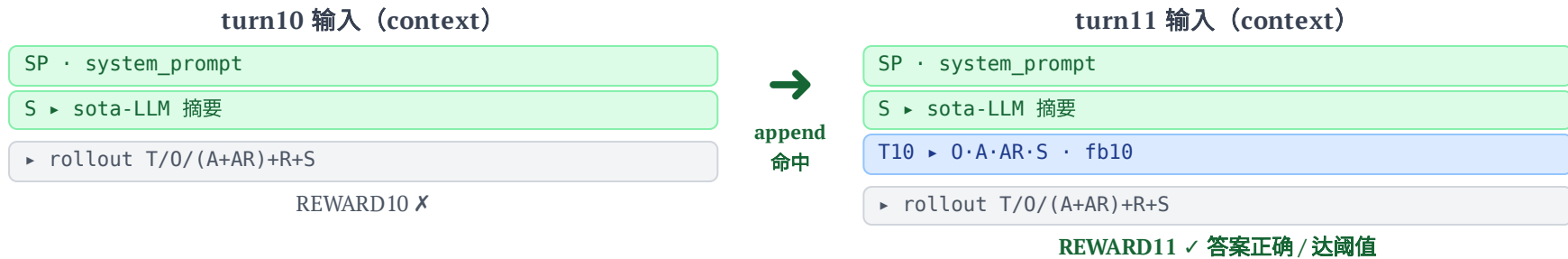
Stage1 收集 S 后仍超上限 → **Stage2** 调 sota-LLM 把 FUP + 所有 S + fb9 摘要成单个 <turn_summary>；前缀只剩 SP。仅此极端情形需一次 LLM 调用，绝大多数压缩是零成本的 Stage1。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · SCRIBE 上下文管理机制 - turn10 → turn11 (终止)



整段复用 + 追加 turn10 ; turn11 提交答案正确 (或 REWARD11 达阈值) → **loop 终止** ✓。整条轨迹: 12 个 turn, 仅 5 次压缩事件, 其余 6 次过渡为纯 append (KV-cache 命中)。

绿 = KV-cache 复用

蓝 = 新增需计算

橙 = 压缩/重算 (删除线=被丢弃)

05 · 上下文机制定义（1/2） - Block 模型与压缩规则

Block 标签与顺序

T=think · A=tool_call · O=output · R=reflect · S=turn_summary ·

AR=tool_response

一个 turn 的 rollout = list[T/O/(list[A]+list[AR])] + R + S

step = 一次 LLM call；N 个 tool_call → N+1 步；最后非工具步产出 O+R+S

结束 turn

（注：rollout 实际输入始终排除 T&R——它们一次性；此处列出全部 block 是为训练时完整记录）

Block 层 vs 消息层

标签在 **block 层** 始终保留（供解析器切分 ScribeBlock）

消息层（模型下次实际读到的）：

- <think>/<reflect> 永不出现在未来输入（一次性）
- <turn_summary> 保留标签（recent 与压缩后 old 都留）
- <tool_call> 在无 S 的 old turn 降级为 O+A 时保留
- OUTPUT 裸文本；AR 由 role=tool 承载

压缩三规则

- ① **T&R 一次性**：生成后在下一步前剥离，永不进未来输入
- ② **recent-k 保留**：recent-k（默认 k=3）之前的 turn 只保留 S（标签完整）
- ③ **turn 原子性**：压缩只在 rollout 前触发（check-before-rollout），不在 turn 内中途触发——赌单 turn 不超长；首个大 turn 也难超当今 1M context

无 S 兜底

若某 turn 无 S（rollout 格式错）：

- 该 turn 老到只需保留 S 时 → 改保留 O + A
（O 作裸文本，A 带 <tool_call> 标签）
 - recent 但无 S 的 turn → 保留 O/A/AR
- 下文示例均假设每 turn 正确生成 S

05 · 上下文机制定义 (2/2) - 压缩集合、两阶段与硬边界

两个集合 (关键区分)

Token-count 集合 (决定「是否压缩」): 整个输入 context = system_prompt + first_user_prompt + 所有 turn-feedback + 所有 input blocks, 一起计数 (**SP 也算**)

Compression 集合 (决定「压缩什么」): 分阶段增长。SP 永不压缩 (逐字保留——承载 task/protocol, 须跨轨迹一致)

硬边界 (Hard Boundary)

若不可压缩前缀 = SP + FUP + 所有 feedbacks (所有 input block 已压成空) 自身仍超上限 → 压缩无能为力
此时 SCRIBE 抛错: 轨迹配置不可行 (逐字保留部分单独溢出 context window)

两阶段压缩

Stage1 (收集 S): 只动 input blocks, 旧 turn 折叠为 S; FUP + feedbacks 逐字保留

Stage2 (sota-LLM, 当 S 序列仍接近上限时触发): LLM 输入包含 FUP + feedbacks, 一起折叠进单个 <turn_summary>; 返回

`{"summary": "..."}` , 校验为恰好一个 turn_summary 块

仅 SP 逐字穿过两阶段; 压缩直到不再接近上限才 rollout; 单块仍超则重试 sota-LLM (其重试/退避与 SCRIBE 解耦)

check-before-rollout 纪律

压缩是 **turn 级** (非 turn 内级):

- 整个 input blocks 的 token 接近上限 → rollout 前自动触发压缩
 - 压缩到不再接近上限 → 才进入 rollout
 - 该 turn 最终存储: 压缩后 input + 新生成/rollout blocks + reward
- 权衡: 原子 turn 的 rollout 是对「该 turn 不至过长使 context 崩」的合理下注

06 · reward mechanism of a turn

16 个 metric (多维 reward 防 hacking) + 权重配比

06 · the reward mechanism of a turn - Metric 1

M1

最终步奖励

final-step reward

w = 0.52

核心信号 · answer correctness

判定：环境（无 judge）

衡量：本 turn 最终步的答案质量与终止方式

机制： $\text{reward} = \text{correctness}(1a) \times \text{natural_termination}(1b) + \text{truncation_penalty}(1c)$

- **1a 答案正确性：**提交答案 == 正确答案 → 1.0，否则 0.0（二元）
- **1b 自然终止：**必须以最终非工具 LLM call 结束（`terminated && done_reason=="no_tool_call"`），否则该因子 0
- **1c 截断惩罚：**被步数上限截断 → -0.2

正交设计：格式由 M4 管，M1 不查块格式 → 便于消融

仅「干净最终非工具步产出的正确答案」拿满分；原始区间 $[-0.2, 1]$ 仿射缩放到 $[0, 1]$

06 · the reward mechanism of a turn - Metric 2

M2

工具使用质量

tool-usage quality

w = 0.08

基础设施 · infrastructure

判定：环境（无 judge）

衡量：本 turn rollout 的工具使用质量

机制： $\text{score} = 1 - 2a \text{ 惩罚} - 2b \text{ 惩罚}$

- **2a submit 恰好一次：**偏离则 $-0.15 / \text{单位}$
- **2b 同一消息内并行相同工具调用**（name + 归一化 args 相同）：每个重复 -0.10

跨步重复不罚（环境状态可能已变：run → edit → run）

阶梯式扣分，单个小错只削分不归零

06 · the reward mechanism of a turn - Metric 3

M3

步数长度

step length

w = 0.02

效率 · efficiency

纯统计 (无 judge)

衡量：步数长度 (A 块计数)

机制： $\text{score} = 1 / (1 + n_a)$

目标：在保持 M1 同水平的同时，让 M3 尽可能低

更少 tool_call 步 → 更高分 (更高效)

纯统计；与 M5 (token 量) 共同构成效率维度

06 · the reward mechanism of a turn - Metric 4

M4

格式正确性

format correctness

w = 0.10

守门员 · gatekeeper

判定：解析器（无 judge）

衡量：格式正确性（守门员）

机制： $\text{score} = 1 - \sum \text{违约惩罚}$ ，阶梯式扣分

- **4a** 期望块类型齐全（T/O/A/AR/R/S）
- **4b** 最终非工具步以 R 再 S 结尾（倒数第二 R，最后 S）
- **4c** R/S 只在最终步出现
- **4d** 最终步不含 tool_call
- **4e** 仅允许约定 SCRIBE 标签（禁 <submit>/<bash>/<plan>...）
- **4f** R/S 必须在 submit 之后
- **4g** 解析无效份额（嵌套标签 / 坏 JSON）

06 · the reward mechanism of a turn - Metric 5

M5

rollout 总 token

total rollout tokens

w = 0.02

效率 · efficiency

纯统计 (无 judge)

衡量：本 turn rollout 总 token (T/O/A/R/S, **AR 除外**)

机制：score = ref / (n + ref), ref = 8192

更多 token → 更低分；**软衰减**，不骤降至 0

对正常长度 rollout 信号常驻；不应主导 reward

06 · the reward mechanism of a turn - Metric 6

M6

token 复用率

token reuse ratio

w = 0.02

summary 防抄袭 · 符号性
纯统计 (无 judge)

衡量：summary 对 ground-truth 关键 token 的复用率

机制：reuse = $|S \cap (O \cup A)| / |O \cup A|$

S = turn_summary token 集；分母仅 O ∪ A (S 故意排除，避免被自身长度膨胀)

衡量「summary 复用了多少 ground-truth 关键 token」；纯统计

06 · the reward mechanism of a turn - Metric 7

M7

忠实度

faithfulness

w = 0.02

summary 质量 · judge

LLM judge (7-10 合一调用)

衡量：summary 是否如实报告 O/A（忠实度）

机制：LLM judge 对照 ground-truth(O+A) 评分

高分 = 准确报告所思所做；低分 = 捏造 / 遗漏 / 矛盾

M7-M10 合并在一次 LLM 调用返回四维分数

06 · the reward mechanism of a turn - Metric 8

M8

方向中立

direction neutrality

w = 0.02

summary 质量 · judge

LLM judge (7-10 合一调用)

衡量：summary 是否保持回顾性（方向中立）

机制：LLM judge；summary 只能回头看本 turn，**不得预测 / 规划下一 turn**

高分 = 纯回顾；低分 = 泄漏到未来规划

保证 S 是「记录」而非「指令」，避免污染下游 turn

06 · the reward mechanism of a turn - Metric 9

M9

turn 聚焦

turn focus

w = 0.02

summary 质量 · judge

LLM judge (7-10 合一调用)

衡量：summary 是否只描述本 turn (turn focus)

机制：LLM judge；严格限定当前 turn，不得混入更早 turn 内容
(后续 turn 内容构造上不会出现；此维只防「前序 turn」渗入)

高分 = 仅本 turn；低分 = 折入前序 turn 行为

06 · the reward mechanism of a turn - Metric 10

M10

流畅度

fluency

w = 0.02

summary 质量 · judge

LLM judge (7-10 合一调用)

衡量：summary 流畅度

机制：LLM judge；防止为堆高 M6 复用率而生硬堆砌关键词

高分 = 自然流畅；与 M6 协同（既要高复用又要可读）

reflect 块是让后续 turn_summary 拿更高 reward 的 CoT 过程

06 · the reward mechanism of a turn - Metric 11

M11

压缩比

compression ratio

w = 0.01

summary 防抄袭 · 符号性

纯统计 (无 judge)

衡量：summary 压缩比

机制：ratio = len(S tokens) / len(OUA tokens) ; score = 1 - ratio

防止把整段 ground-truth 当 summary (会拖垮 M11)

纯统计；与 M6 (用集合) 不同，此处用**长度**

06 · the reward mechanism of a turn - Metric 12

M12

跨块 n-gram 重叠

cross-block n-gram overlap

w = 0.03

summary 防抄袭 · anti copy-paste

纯统计 (无 judge)

衡量：跨块 n-gram 重叠 (防 copy-paste)

机制： $n=4$; $\text{overlap} = |\text{shared}| / |\text{ngrams}(S)|$; $\text{score} = 1 - \text{overlap}$

#11 只挡整段照搬, #12 抓「部分逐字搬运」(连续短语) -- 能绕过 #11、甚至骗过 #10

纯统计, 廉价

06 · the reward mechanism of a turn - Metric 13

M13

块内 n-gram 重叠

intra-block n-gram overlap

w = 0.03

summary 防抄袭 · anti loop

纯统计 (无 judge)

衡量：块内 n-gram 重叠 (防块内自循环)

机制：对 T/O/R/S 块 (A 结构化天然重复、AR 环境产出，均排除)

$\text{intra} = (\text{total_ngrams} - \text{unique_ngrams}) / \text{total_ngrams}$; token 加权均值 ; $\text{score} = 1 - \text{penalty}$

#12 只抓跨块抄袭，看不见块内自我重复 (小模型 / 早期 RL 常见退化)

06 · the reward mechanism of a turn - Metric 14

M14

工具参数合法性

malformed tool-call penalty

w = 0.02

基础设施 · infrastructure

判定：执行层（无 judge）

衡量：工具调用参数合法性

机制：每个 malformed_arguments / exec_error 调用 -0.10；clamp [0,1]

例：JSON 不可解析、args 非对象、缺必填参（bash.command）、多出幻觉参（bash.stdout）

完美 = 1.0；给 GRPO **连续梯度**抑制，而非任其循环到截断

06 · the reward mechanism of a turn - Metric 15

M15

跨步重复工具调用

repeated tool-call penalty

w = 0.02

基础设施 · infrastructure

判定：执行层（无 judge）

衡量：跨步重复工具调用

机制：每个与本 turn 早先执行过的 (name, 归一化 args) 完全相同的调用 -0.08；clamp [0,1]

例：bash{ls} → 结果 → bash{ls}；与 M2b（单消息内并行重复）互补

完美 = 1.0；与 M14 一起止住「重复 / 坏调用浪费步数到截断」

06 · the reward mechanism of a turn - Metric 16

M16

跨 turn 重复提交

cross-turn duplicate submit

w = 0.05

防停滞 · anti stalling (唯一跨 turn)

判定: history_manager (无 judge)

衡量: 跨 turn 重复提交相同答案 (唯一跨 turn 惩罚)

机制: 三条件全满足才触发: ①本 turn submit ≥ 1 ②存在上一 turn 答案 ③本 turn 答案 == 上一 turn 答案

触发: $1.0 - 0.20$; 否则 1.0 (含两次答案不同--正是「用反馈改进」)

防「忽略反馈、逐 turn 重提相同答案」的 stalling; intra-turn 指标(2a/2b/15)看不见
prev_turn_answer 由 history_manager 在 finalize 时读出

06 · the reward mechanism of a turn - 16 metric 权重配比总结

M	metric	w	类别
M1	最终步奖励 answer+终止	0.52	核心
M2	工具使用质量	0.08	基础设施
M3	步数长度	0.02	效率
M4	格式正确性	0.10	守门员
M5	rollout 总 token	0.02	效率
M6	token 复用率	0.02	防抄袭
M7	忠实度 judge	0.02	summary 质量
M8	方向中立 judge	0.02	summary 质量
M9	turn 聚焦 judge	0.02	summary 质量
M10	流畅度 judge	0.02	summary 质量
M11	压缩比	0.01	防抄袭
M12	跨块 n-gram 重叠	0.03	防抄袭
M13	块内 n-gram 重叠	0.03	防抄袭
M14	工具参数合法	0.02	基础设施
M15	跨步重复工具	0.02	基础设施
M16	跨 turn 重复提交	0.05	防停滞
合计 Σw_i		1.00	clamp [0,1]

类别小计

- 核心 answer : M1 = **0.52**
- 守门员 format : M4 = **0.10**
- 基础设施 tool : M2+M14+M15 = **0.12**
- 效率 : M3+M5 = **0.04**
- summary 质量(judge) : M7-M10 = **0.08**
- summary 防抄袭(统计) : M6+M11+M12+M13 = **0.09**
- 防停滞 : M16 = **0.05**

设计哲学

- answer 主导但不过激 (0.52)
- 防抄袭指标为符号/反抄袭信号，非主驱动
- **多维 reward 防 hacking**
- M7-M10 合并一次 LLM 调用
- total = clamp($\Sigma w_i \cdot m_i$, 0, 1)

trajectory reward

```
trajectory_reward = mean(turn_reward) ×  
decay^len(turns)
```

用 mean 而非仅末 turn : 早 turn 坏行为直接受罚 ; 长度衰减奖励更少 turn 解决

07 · abstract of our contribution

可作为 A 类会议（如 ICLR）论文摘要

07 · abstract of our contribution

We present **SCRIBE**, a small yet foundational change to the ReAct loop for long-horizon **loop-engineering** tasks: at the end of every turn the agent explicitly writes a **Reflect (R)** and a **Turn-Summary (S)** of what it just did. This single change induces a new **context-management mechanism** — disposable T/R blocks, recent- k full retention, and automatic **Stage-1** (collect-S) / **Stage-2** (sota-LLM) compression — which in turn unlocks four benefits:

01

Painless context extension

Context grows across turns yet stays bounded, almost entirely by **zero-cost automatic compression** — only the rare Stage-2 needs one LLM call.

02

Token-level credit assignment

Each turn's summary is a first-class, rewarded block, so GRPO credit flows **trajectory → turn → step → token** — markedly more sample-efficient than broad-ReAct.

03

Higher accuracy & token efficiency

Both accuracy and token efficiency on loop-engineering tasks **exceed the pre-training model and the ReAct-trained model**.

04

High KV-cache reuse

Between consecutive turns only a small suffix is new while the **whole prefix is cache-reused**; compression is the only prefix-breaking event.

A small & beautiful change → a context-mechanism shift → many derived benefits.



谢谢聆听

Q & A

周浩洋 · JLU, SE · 2025.04.17 – Present